Process Synchronization ❁

Process synchronization is a set of techniques used to coordinate the execution of multiple processes that share resources. The main goal is to prevent **race conditions** and data inconsistency by ensuring that critical sections of code are executed in an atomic manner.

There are two main types of processes in the context of synchronization:

1. Independent Processes

An **independent process** does not share any data or resources with other processes. The execution of one independent process does not affect the execution of others. Such processes do not require synchronization mechanisms because there is no risk of data corruption or resource contention.

2. Cooperating Processes

A **cooperating process** can affect or be affected by other processes. They share data and resources, making their execution interdependent. These processes require synchronization to prevent issues like deadlocks and data races. Most of the challenges in process synchronization arise from cooperating processes.

Race Condition

A **race condition** is a situation that occurs when multiple processes access and manipulate the same shared data concurrently, and the final outcome depends on the unpredictable order of execution. This can lead to incorrect results, as the output is determined by which process finishes its operations last.

Example: Bank Account Transaction

Imagine a bank account with a balance of $100. Two processes, P1 and P2, try to perform a transaction simultaneously.

- **P1**: Deposits $50.
- **P2**: Withdraws $20.

The correct final balance should be $130.

Execution Order 1 (Expected):

1. **P1** reads the balance (100).
2. **P1** adds $50 (result is 150).
3. **P1** writes the new balance (150).
4. **P2** reads the balance (150).
5. **P2** subtracts $20 (result is 130).
6. **P2** writes the new balance (130).

- **Final Balance**: $130 (Correct)

  Execution Order 2 (Race Condition):

1. **P1** reads the balance (100).
2. **P2** reads the balance (100).
3. **P1** adds $50 (result is 150).
4. **P2** subtracts $20 (result is 80).
5. **P1** writes the new balance (150).
6. **P2** writes the new balance (80).

- **Final Balance**: $80 (Incorrect)

  In this race condition, the final balance depends on which process writes the data last. Process P2's final write overwrites the result of P1's operation, leading to a loss of $50. This is why synchronization is crucial to ensure that shared resources are accessed in a controlled, serial manner.